

go-zero 微服务框架知识点总结

- 1. 环境准备与安装
 - 1.1. Go 环境配置
 - 1.2. go-zero 与 goctl 安装
 - 1.3. 开发工具配置
- 2. 快速入门
 - 2.1. 创建 API 项目
 - 2.2. 项目目录结构
 - 2.3. 核心架构流程
- 3. API 服务开发
 - 3.1. API 定义文件
 - 3.2. 代码生成与开发流程
 - 3.3. 业务逻辑层
 - 3.4. 服务上下文与依赖注入
- 4. RPC 服务开发
 - 4.1. 创建 RPC 项目
 - 4.2. Protobuf 定义
 - 4.3. RPC 服务实现
 - 4.4. 服务调用与测试
- 5. 数据库操作
 - 5.1. 模型生成
 - 5.2. CRUD 操作
 - 5.3. 高级查询
 - 5.4. 事务处理
- 6. 配置与日志
 - 6.1. 配置文件结构
 - 6.2. 日志系统
 - 6.3. 多环境管理
- 7. 缓存系统
 - 7.1. Redis 配置
 - 7.2. 缓存策略
 - 7.3. 缓存最佳实践
- 8. 中间件与拦截器
 - 8.1. 自定义中间件
 - 8.2. 内置中间件
 - 8.3. 认证与鉴权
 - 8.4. 统一响应处理
- 9. 服务治理
 - 9.1. 超时与重试
 - 9.2. 限流
 - 9.3. 熔断
 - 9.4. 降级
 - 9.5. 负载均衡与健康检查
- 10. 错误处理与响应规范

- 10.1. 错误类型定义
 - 10.2. 统一响应格式
 - 10.3. 错误处理最佳实践
 - 11. 常用功能
 - 11.1. 文件上传下载
 - 11.2. ID 生成器
 - 11.3. 定时任务
 - 11.4. 分布式锁
 - 11.5. 事务处理
 - 11.6. 错误处理
 - 12. 部署与运维
 - 12.1. Docker 部署
 - 12.2. Kubernetes 部署
 - 12.3. 性能测试
 - 13. 微服务架构
 - 13.1. 服务拆分原则
 - 13.2. 服务间通信
 - 14. 调试与实战
 - 14.1. 调试接口
 - 14.2. 用户注册流程
 - 14.3. 支付订单流程
 - 15. 附录
 - 15.1. goctl 命令速查
 - 15.2. 常见问题
 - 15.3. 最佳实践
 - 16. 总结
-

1. 环境准备与安装

1.1. Go 环境配置

go-zero 需要 Go 1.18 及以上版本，推荐使用 1.21+。

安装 Go：

```
# macOS/Linux
wget https://go.dev/dl/go1.21.5.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.21.5.linux-amd64.tar.gz

# 配置环境变量
export PATH=$PATH:/usr/local/go/bin
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin

# 验证安装
go version
```

配置 Go 模块代理（国内推荐）：

```
go env -w GOMODCACHE=on
go env -w GOPROXY=https://goproxy.cn,direct
go env -w GOSUMDB=sum.golang.google.cn
```

1.2. go-zero 与 goctl 安装

安装 go-zero：

```
# 安装 go-zero 框架
go get -u github.com/zeromicro/go-zero@latest
```

安装 goctl 工具：

```
# 安装 goctl（代码生成工具）
go install github.com/zeromicro/go-zero/tools/goctl@latest

# 验证安装
goctl --version
```

安装 protoc 及相关依赖：

```
# 一键安装 protoc、protoc-gen-go、protoc-gen-go-grpc
goctl env check -i -f --verbose

# 验证安装
protoc --version
```

手动安装 protoc（可选）：

```
# macOS
brew install protobuf

# Linux
# 下载并安装 protoc
wget
https://github.com/protocolbuffers/protobuf/releases/download/v25.1/protoc-25.1-linux-x86_64.zip
unzip protoc-25.1-linux-x86_64.zip -d /usr/local

# 安装 Go 插件
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

1.3. 开发工具配置

IDE 插件推荐：

工具	插件	说明
VSCode	go	Go 语言官方插件
VSCode	gocli	go-zero 语法高亮
GoLand	Go	JetBrains Go IDE
GoLand	go-zero	go-zero 插件

VSCode 配置示例：

```
// settings.json
{
  "go.useLanguageServer": true,
  "go.lintTool": "golangci-lint",
  "go.lintOnSave": "package",
  "editor.formatOnSave": true,
  "go.formatTool": "goimports"
}
```

小节：

- ✓ Go 环境配置：安装 Go 1.21+ 并配置模块代理
- ✓ gocli 安装：代码生成工具是 go-zero 开发核心
- ✓ protoc 安装：RPC 开发必需，可一键安装
- ✓ 开发工具：推荐使用 VSCode + gocli 插件

2. 快速入门

2.1. 创建 API 项目

创建第一个 API 服务：

```
# 创建项目
gocli api new myapi
cd myapi

# 初始化依赖
go mod tidy

# 启动服务
go run myapi.go -f etc/myapi-api.yaml
```

```
# 访问测试
curl http://localhost:8888/from/you
# 响应: {"from":"you"}
```

创建 RPC 服务：

```
# 创建 RPC 项目
goctl rpc new myrpc
cd myrpc

# 初始化并启动
go mod tidy
go run myrpc.go -f etc/myrpc.yaml
```

2.2. 项目目录结构

API 项目结构：

```
myapi/
├── etc/                # 配置文件目录
│   └── myapi-api.yaml  # YAML 配置文件
├── internal/          # 内部代码（不对外暴露）
│   ├── config/        # 配置定义
│   │   └── config.go   # 配置结构体
│   ├── handler/       # 路由处理器
│   │   ├── routes.go   # 路由注册
│   │   └── myapi_handler.go # Handler 实现
│   ├── logic/         # 业务逻辑层
│   │   └── myapi_logic.go # Logic 实现
│   ├── svc/           # 服务上下文
│   │   └── servicecontext.go # 依赖注入容器
│   └── types/         # 类型定义
│       └── types.go     # 请求/响应结构体
├── myapi.api          # API 定义文件（DSL）
└── myapi.go           # 程序入口
```

各目录职责：

目录	说明	职责
etc/	配置文件	存放 YAML 配置，支持多环境
config/	配置结构	定义配置结构体，自动映射 YAML
handler/	路由处理	接收 HTTP 请求，参数校验，调用 Logic
logic/	业务逻辑	核心业务代码，不包含 HTTP 细节
svc/	服务上下文	依赖注入容器，管理 Model、RPC 等

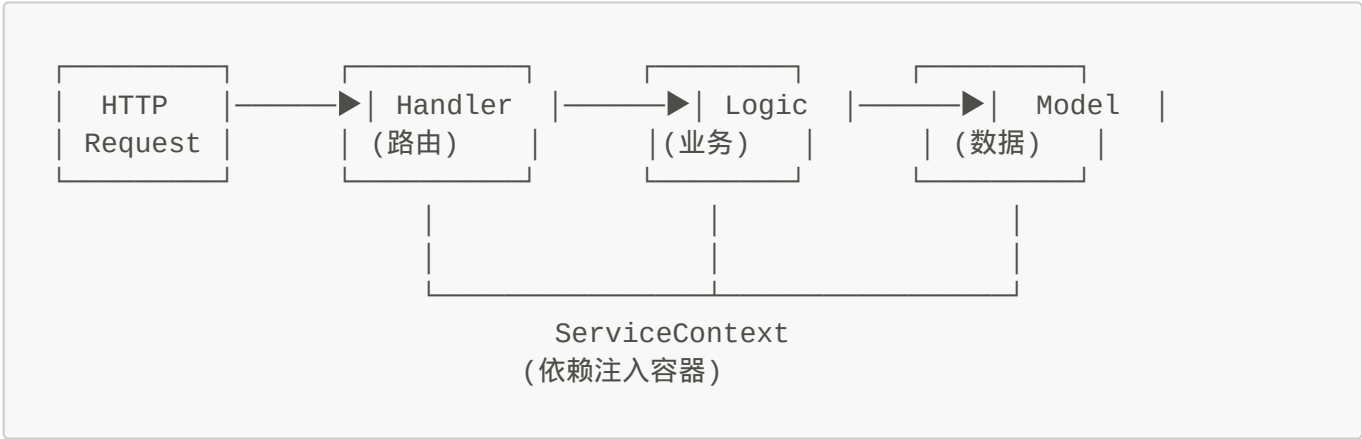
目录	说明	职责
types/	类型定义	API 请求和响应结构体
model/	数据模型	数据库操作（需单独生成）

RPC 项目结构：

```
myrpc/
├── etc/
│   └── myrpc.yaml           # RPC 配置
├── internal/
│   ├── config/             # 配置定义
│   ├── server/             # RPC 服务实现
│   │   └── myrpc_server.go
│   ├── svc/                # 服务上下文
│   └── logic/              # 业务逻辑
├── pb/                     # Protobuf 生成的代码
│   └── myrpc.pb.go
├── myrpc.proto              # Protobuf 定义文件
└── myrpc.go                 # 程序入口
```

2.3. 核心架构流程

请求处理流程：



各层职责详解：

层级	职责	关键点
Handler	接收请求、参数解析、调用 Logic	不写业务逻辑，仅转发
Logic	业务逻辑处理、数据编排	核心代码所在，可调用多个 Model/RPC
Model	数据库 CRUD 操作	单表操作，复杂查询可用 QueryBuilder
ServiceContext	管理依赖、注入资源	Model、RPC、Redis 等都在此初始化

核心设计理念：

1. **分层清晰**：Handler → Logic → Model，职责单一
2. **依赖注入**：通过 ServiceContext 管理所有依赖
3. **代码生成**：gocli 自动生成框架代码，开发者专注业务
4. **工程化**：内置日志、监控、限流、熔断等

小节：

- ✓ 创建项目：使用 `gocli api new` 快速创建 API 项目
- ✓ 目录结构：清晰的分层架构，职责明确
- ✓ 架构流程：Handler → Logic → Model，依赖注入管理资源

3. API 服务开发

3.1. API 定义文件

go-zero 使用 `.api` 文件定义 HTTP API，采用 DSL（领域特定语言）语法，清晰描述接口规范。

基础语法：

```
// user.api - 用户服务 API 定义

syntax = "v1" // API 版本

info (
    title:    "用户服务"           // 服务标题
    desc:     "用户相关接口"       // 服务描述
    author:   "yourname"          // 作者
    version:  "1.0"               // 版本
)

// 定义请求类型
type (
    // 获取用户请求
    GetUserRequest {
        Id int64 `path:"id"` // 路径参数
    }

    // 用户信息响应
    UserResponse {
        Id      int64 `json:"id"`
        Name    string `json:"name"`
        Email   string `json:"email"`
        Status  int    `json:"status,default=1"`
    }

    // 创建用户请求
    CreateUserRequest {
        Name      string `json:"name"`           // 必填
        Email     string `json:"email,optional"` // 可选
        Password  string `json:"password,minlength=6,maxlength=20"`
    }
}
```

```
// 统一响应
Response {
    Code    int           `json:"code"`
    Message string        `json:"message"`
    Data    interface{}   `json:"data,optional"`
}

// 服务定义
@server(
    prefix: /api/v1           // URL 前缀
    group:  user              // 路由分组 (代码也会分组)
    jwt:    Auth              // JWT 认证
    middleware: AuthMiddleware // 中间件
)
service user-api {
    @doc "获取用户信息"
    @handler getUser
    get /user/:id (GetUserRequest) returns (UserResponse)

    @doc "创建用户"
    @handler createUser
    post /user (CreateUserRequest) returns (Response)

    @doc "更新用户"
    @handler updateUser
    put /user/:id (UpdateUserRequest) returns (Response)

    @doc "删除用户"
    @handler deleteUser
    delete /user/:id (GetUserRequest) returns (Response)
}
```

常用语法标签：

标签	说明	示例	使用场景
path:"name"	路径参数	/user/:id	RESTful 接口
query:"page"	查询参数	?page=1	分页、过滤
header:"token"	请求头	Header 中获取	认证 token
json:"name"	JSON 字段	{"name": "xxx"}	POST/PUT 请求体
form:"name"	表单字段	name=xxx	Form 表单
optional	可选字段	json:"name,optional"	非必填
required	必填字段	json:"name"	默认必填
default=1	默认值	default=1	参数默认值

标签	说明	示例	使用场景
options=a\ b	枚举值	options=male\ female	限定取值范围
range=[1,100]	数值范围	range=[1,100]	数值约束
min=2,max=20	长度限制	字符串长度 2-20	字符串校验
email	邮箱格式	邮箱校验	邮箱字段
mobile	手机号格式	手机号校验	手机号字段

多服务定义：

```
// 定义多个服务 ( API + RPC )
service user-api {
    @handler getUser
    get /user/:id (GetUserRequest) returns (UserResponse)
}

service user-rpc {
    @handler getUserRpc
    post /rpc/user/get (GetUserRequest) returns (UserResponse)
}
```

3.2. 代码生成与开发流程

开发流程：

```
编写 .api 文件 → goctl 生成代码 → 实现 Logic → 测试验证
```

代码生成命令：

```
# 方式1：创建新项目（推荐）
goctl api new myapi
cd myapi
# 编辑 myapi.api 文件，定义接口
goctl api go -api myapi.api -dir . # 重新生成代码

# 方式2：已有 .api 文件，生成代码
goctl api go -api user.api -dir .

# 方式3：指定生成的 Go 文件
goctl api go -api user.api -go user.go

# 格式化 API 文件
goctl api format -dir user.api
```

```
# 验证 API 文件语法
goctl api validate -api user.api
```

生成的代码结构：

```
// types/types.go - 自动生成
type GetUserRequest struct {
    Id int64 `path:"id"`
}

type UserResponse struct {
    Id      int64 `json:"id"`
    Name    string `json:"name"`
    Email   string `json:"email"`
    Status  int    `json:"status"`
}

// handler/user/get_user_handler.go - Handler 层
func GetUserHandler(svcCtx *svc.ServiceContext) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        var req types.GetUserRequest
        if err := httpx.Parse(r, &req); err != nil {
            httpx.Error(w, err)
            return
        }

        l := logic.NewGetUserLogic(r.Context(), svcCtx)
        resp, err := l.GetUser(&req)
        if err != nil {
            httpx.Error(w, err)
        } else {
            httpx.OkJson(w, resp)
        }
    }
}

// logic/user/get_user_logic.go - Logic 层 (需要实现)
func (l *GetUserLogic) GetUser(req *types.GetUserRequest) (resp *types.UserResponse, err error) {
    // TODO: 实现业务逻辑
    return &types.UserResponse{
        Id:    req.Id,
        Name:  "张三",
    }, nil
}
```

最佳实践：

1. 先设计 API，后写代码：使用 .api 文件作为接口文档
2. 版本化管理 API：使用 v1.api、v2.api 管理版本

3. 分组管理：使用 `group` 参数对接口分组，代码结构清晰
4. 不要修改生成的代码：修改 `.api` 文件后重新生成

3.3. 业务逻辑层

Logic 层是业务核心，不包含 HTTP 相关代码，便于测试和复用。

Logic 结构：

```
// internal/logic/user/get_user_logic.go
type GetUserLogic struct {
    logx.Logger
    ctx      context.Context
    svcCtx   *svc.ServiceContext
}

func NewGetUserLogic(ctx context.Context, svcCtx *svc.ServiceContext)
*GetUserLogic {
    return &GetUserLogic{
        Logger: logx.WithContext(ctx),
        ctx:    ctx,
        svcCtx: svcCtx,
    }
}

func (l *GetUserLogic) GetUser(req *types.GetUserRequest) (resp
*types.UserResponse, err error) {
    // 1. 参数校验（简单校验已在 Handler 完成）
    if req.Id <= 0 {
        return nil, errors.New("用户ID无效")
    }

    // 2. 查询数据库
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        if err == model.ErrNotFound {
            return nil, errors.New("用户不存在")
        }
        return nil, err
    }

    // 3. 数据转换
    return &types.UserResponse{
        Id:      user.Id,
        Name:    user.Name,
        Email:   user.Email,
        Status:  user.Status,
    }, nil
}
```

复杂业务示例：

```
// 创建订单 Logic
func (l *CreateOrderLogic) CreateOrder(req *types.CreateOrderRequest)
(*types.OrderResponse, error) {
    // 1. 使用分布式锁
    lockKey := fmt.Sprintf("lock:order:%d", req.UserId)
    lock, err := redis.Lock(l.svcCtx.Redis, lockKey, 10*time.Second)
    if err != nil {
        return nil, errors.New("系统繁忙，请稍后重试")
    }
    defer lock.Unlock()

    // 2. 查询商品信息
    product, err := l.svcCtx.ProductModel.FindOne(l.ctx, req.ProductId)
    if err != nil {
        return nil, err
    }

    // 3. 检查库存
    if product.Stock < req.Quantity {
        return nil, errors.New("库存不足")
    }

    // 4. 创建订单（事务）
    var orderId int64
    err = l.svcCtx.Transact(func(session sqlx.Session) error {
        // 扣减库存
        if err := l.svcCtx.ProductModel.DecrStock(l.ctx, req.ProductId,
req.Quantity); err != nil {
            return err
        }

        // 创建订单
        order := &model.Order{
            UserId:    req.UserId,
            ProductId: req.ProductId,
            Quantity:  req.Quantity,
            Amount:    product.Price * int64(req.Quantity),
            Status:    0, // 待支付
        }
        result, err := l.svcCtx.OrderModel.Insert(l.ctx, order)
        if err != nil {
            return err
        }
        orderId, _ = result.LastInsertId()
        return nil
    })

    if err != nil {
        return nil, err
    }

    return &types.OrderResponse{
```

```
        OrderId: orderId,
        Message: "下单成功",
    }, nil
}
```

Logic 层最佳实践：

实践	说明	示例
使用 Context	始终传递 <code>l.ctx</code>	<code>Model.FindOne(l.ctx, id)</code>
错误处理	返回明确的错误信息	<code>errors.New("用户不存在")</code>
日志记录	使用 <code>l.Logger</code> 记录关键信息	<code>l.Infof("用户登录: %d", userId)</code>
依赖注入	通过 <code>l.svcCtx</code> 获取依赖	<code>l.svcCtx.UserModel</code>
事务处理	使用 <code>Transact</code> 方法	<code>l.svcCtx.Transact(func() {})</code>

3.4. 服务上下文与依赖注入

ServiceContext 是依赖注入容器，管理所有资源。

定义 ServiceContext：

```
// internal/svc/servicecontext.go
type ServiceContext struct {
    Config  config.Config
    Redis   *redis.Redis

    // 数据模型
    UserModel  model.UserModel
    OrderModel model.OrderModel

    // RPC 客户端
    UserRpc   user.UserClient
    OrderRpc  order.OrderClient

    // 其他服务
    Trans     *sqlx.Tx    // 事务管理器
}

func NewServiceContext(c config.Config) *ServiceContext {
    // 初始化数据库连接
    conn := sqlx.NewMysql(c.Mysql.DataSource)

    return &ServiceContext{
        Config:      c,
        Redis:       redis.New(c.Redis.Host),
        UserModel:   model.NewUserModel(conn),
        OrderModel:  model.NewOrderModel(conn),
        UserRpc:
    }
```

```

user.NewUserClient(zrpc.MustNewClient(c.UserRpc).Conn()),
    Trans:      conn, // 事务管理
}
}

```

注入 RPC 客户端：

```

# etc/myapi-api.yaml
Name: myapi-api
Host: 0.0.0.0
Port: 8888

# RPC 服务配置
UserRpc:
  Etcd:
    Hosts:
      - localhost:2379
    Key: user.rpc

OrderRpc:
  Etcd:
    Hosts:
      - localhost:2379
    Key: order.rpc

```

```

// config/config.go
type Config struct {
    rest.RestConf
    UserRpc zrpc.RpcClientConf // RPC 配置
    OrderRpc zrpc.RpcClientConf
}

// 使用 RPC 客户端
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    // 调用 RPC 服务
    resp, err := l.svcCtx.UserRpc.GetUser(l.ctx, &user.IdRequest{
        Id: req.Id,
    })
    if err != nil {
        return nil, err
    }

    return &types.UserResponse{
        Id: resp.Id,
        Name: resp.Name,
        Email: resp.Email,
    }, nil
}

```

依赖注入最佳实践：

1. **集中管理依赖**：所有依赖在 ServiceContext 中初始化
2. **懒加载**：某些依赖可以懒加载，提高启动速度
3. **配置驱动**：依赖从配置文件读取，支持多环境
4. **接口隔离**：Model 定义接口，便于 Mock 测试

小节：

- ✓ API 定义：使用 .api 文件定义接口，支持丰富的语法标签
- ✓ 代码生成：gocli 自动生成框架代码，专注业务实现
- ✓ Logic 层：业务核心，通过 ServiceContext 获取依赖
- ✓ 依赖注入：ServiceContext 管理所有资源，配置驱动

4. RPC 服务开发

4.1. 创建 RPC 项目

创建 RPC 服务：

```
# 创建 RPC 项目
gocli rpc new user-rpc
cd user-rpc

# 初始化依赖
go mod tidy

# 启动服务
go run user-rpc.go -f etc/user-rpc.yaml
```

RPC 项目结构：

```
user-rpc/
├── etc/
│   └── user-rpc.yaml          # RPC 配置文件
├── internal/
│   ├── config/               # 配置定义
│   │   └── config.go
│   ├── server/               # RPC 服务实现
│   │   └── user_server.go
│   ├── svc/                  # 服务上下文
│   │   └── servicecontext.go
│   └── logic/                 # 业务逻辑
│       └── getuserlogic.go
└── pb/                       # Protobuf 生成的代码
    ├── user.pb.go            # 消息定义
    └── user_grpc.pb.go        # gRPC 服务定义
```

```
|— user.proto          # Protobuf 定义文件
|— user-rpc.go         # 程序入口
```

配置文件：

```
# etc/user-rpc.yaml
Name: user-rpc
ListenOn: 0.0.0.0:8080

# 数据库配置
Mysql:
  DataSource: root:password@tcp(localhost:3306)/dbname?
  charset=utf8mb4&parseTime=true

# Redis 配置
Redis:
  Host: localhost:6379

# Etcd 服务注册（可选）
Etcd:
  Hosts:
    - localhost:2379
  Key: user.rpc
```

4.2. Protobuf 定义

Protobuf 文件：

```
// user.proto
syntax = "proto3";

package user;

option go_package = "./pb";

// 导入 Google 空消息
import "google/protobuf/empty.proto";

// 请求消息
message IdRequest {
  int64 id = 1;
}

message GetUserRequest {
  int64 id = 1;
}

// 响应消息
message UserResponse {
```



```
    int64 id = 1;
    string name = 2;
    string email = 3;
    int32 status = 4;
    int64 created_at = 5;
}

message CreateUserRequest {
    string name = 1;
    string email = 2;
    string password = 3;
}

message CreateUserResponse {
    int64 id = 1;
}

message UpdateUserRequest {
    int64 id = 1;
    string name = 2;
    string email = 3;
}

message ListUsersRequest {
    int32 page = 1;
    int32 page_size = 2;
}

message ListUsersResponse {
    repeated UserResponse users = 1;
    int64 total = 2;
}

// 服务定义
service User {
    // 获取用户
    rpc GetUser(GetUserRequest) returns (UserResponse);

    // 创建用户
    rpc CreateUser(CreateUserRequest) returns (CreateUserResponse);

    // 更新用户
    rpc UpdateUser(UpdateUserRequest) returns (google.protobuf.Empty);

    // 删除用户
    rpc DeleteUser(IdRequest) returns (google.protobuf.Empty);

    // 用户列表
    rpc ListUsers(ListUsersRequest) returns (ListUsersResponse);
}
```

生成代码：

```
# 方式1: 从 .proto 文件生成
goctl rpc protoc user.proto --go_out=. --go-grpc_out=. --zrpc_out=.

# 方式2: 使用 goctl 命令
goctl rpc protoc user.proto -src=. -dir=.

# 只生成 pb 文件 (不生成项目结构)
protoc --go_out=. --go-grpc_out=. user.proto
```

Protobuf 最佳实践：

规范	说明	示例
字段编号	1-15 用于常用字段，节省空间	<code>int64 id = 1;</code>
命名规范	使用驼峰命名	<code>user_name</code> → <code>userName</code>
消息复用	通用消息可复用	<code>IdRequest</code> 用于多个接口
版本管理	使用 package 区分版本	<code>package user.v1;</code>
注释	添加清晰的注释	<code>// 用户ID</code>

4.3. RPC 服务实现

服务端实现：

```
// internal/server/user_server.go
type UserServer struct {
    svcCtx *svc.ServiceContext
    pb.UnimplementedUserServer
}

func NewUserServer(svcCtx *svc.ServiceContext) *UserServer {
    return &UserServer{
        svcCtx: svcCtx,
    }
}

// 获取用户
func (s *UserServer) GetUser(ctx context.Context, req *pb.GetUserRequest) (*pb.UserResponse, error) {
    // 查询数据库
    user, err := s.svcCtx.UserModel.FindOne(ctx, req.Id)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "用户不存在")
    }

    return &pb.UserResponse{
        Id:      user.Id,
        Name:    user.Name,
```

```

        Email:      user.Email,
        Status:     user.Status,
        CreatedAt:  user.CreatedAt.Unix(),
    }, nil
}

// 创建用户
func (s *UserServer) CreateUser(ctx context.Context, req
*pb.CreateUserRequest) (*pb.CreateUserResponse, error) {
    // 参数校验
    if req.Name == "" || req.Email == "" {
        return nil, status.Errorf(codes.InvalidArgument, "参数错误")
    }

    // 创建用户
    user := &model.User{
        Name:      req.Name,
        Email:     req.Email,
        Password:  req.Password,
    }

    result, err := s.svcCtx.UserModel.Insert(ctx, user)
    if err != nil {
        return nil, status.Errorf(codes.Internal, "创建失败")
    }

    userId, _ := result.LastInsertId()
    return &pb.CreateUserResponse{Id: userId}, nil
}

// 更新用户
func (s *UserServer) UpdateUser(ctx context.Context, req
*pb.UpdateUserRequest) (*emptypb.Empty, error) {
    user, err := s.svcCtx.UserModel.FindOne(ctx, req.Id)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "用户不存在")
    }

    user.Name = req.Name
    user.Email = req.Email

    if err := s.svcCtx.UserModel.Update(ctx, user); err != nil {
        return nil, status.Errorf(codes.Internal, "更新失败")
    }

    return &emptypb.Empty{}, nil
}

```

使用 Logic 层（推荐）：

```

// internal/logic/get_user_logic.go
func (l *GetUserLogic) GetUser(in *pb.GetUserRequest) (*pb.UserResponse,

```

```

error) {
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, in.Id)
    if err != nil {
        return nil, err
    }

    return &pb.UserResponse{
        Id:      user.Id,
        Name:    user.Name,
        Email:   user.Email,
    }, nil
}

```

4.4. 服务调用与测试

客户端调用：

```

// 方式1：直接连接
func main() {
    client := user.NewUserClient(zrpc.MustNewClient(zrpc.RpcClientConf{
        Endpoints: []string{"localhost:8080"},
    }).Conn())

    resp, err := client.GetUser(context.Background(),
    &pb.GetUserRequest{Id: 1})
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("User: %+v\n", resp)
}

// 方式2：通过 Etcd 服务发现
client := user.NewUserClient(zrpc.MustNewClient(zrpc.RpcClientConf{
    Etcd: discov.EtcdConf{
        Hosts: []string{"localhost:2379"},
        Key:    "user.rpc",
    },
}).Conn())

// 方式3：在 API 服务中调用（配置文件方式）
// etc/myapi-api.yaml
UserRpc:
  Etcd:
    Hosts:
      - localhost:2379
    Key: user.rpc

// internal/svc/servicecontext.go
func NewServiceContext(c config.Config) *ServiceContext {
    return &ServiceContext{

```

```
        Config: c,
        UserRpc: user.NewUserClient(zrpc.MustNewClient(c.UserRpc).Conn()),
    }
}

// internal/logic/get_user_logic.go
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    resp, err := l.svcCtx.UserRpc.GetUser(l.ctx, &pb.GetUserRequest{Id:
req.Id})
    if err != nil {
        return nil, err
    }

    return &types.UserResponse{
        Id:    resp.Id,
        Name:  resp.Name,
        Email: resp.Email,
    }, nil
}
```

使用 grpcurl 测试：

```
# 安装 grpcurl
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest

# 查看服务列表
grpcurl -plaintext localhost:8080 list

# 查看服务方法
grpcurl -plaintext localhost:8080 list user.User

# 查看方法详情
grpcurl -plaintext localhost:8080 describe user.User.GetUser

# 调用方法
grpcurl -plaintext -d '{"id": 1}' localhost:8080 user.User.GetUser

# 响应示例
{
  "id": "1",
  "name": "张三",
  "email": "zhangsan@example.com",
  "status": 1
}
```

错误处理：

```

import "google.golang.org/grpc/codes"
import "google.golang.org/grpc/status"

// 返回标准 gRPC 错误
func (s *UserServer) GetUser(ctx context.Context, req *pb.GetUserRequest)
(*pb.UserResponse, error) {
    user, err := s.svcCtx.UserModel.FindOne(ctx, req.Id)
    if err != nil {
        if err == model.ErrNotFound {
            return nil, status.Errorf(codes.NotFound, "用户不存在")
        }
        return nil, status.Errorf(codes.Internal, "内部错误")
    }

    return &pb.UserResponse{Id: user.Id}, nil
}

// 客户端处理错误
resp, err := client.GetUser(ctx, req)
if err != nil {
    st, ok := status.FromError(err)
    if ok {
        switch st.Code() {
        case codes.NotFound:
            log.Println("用户不存在")
        case codes.Internal:
            log.Println("内部错误")
        }
    }
}
}

```

小节：

- ✓ RPC 项目：使用 `goctl rpc new` 创建，结构清晰
- ✓ Protobuf：定义消息和服务，支持多种数据类型
- ✓ 服务实现：实现 Server 接口，处理业务逻辑
- ✓ 客户端调用：支持直连和 Etcd 服务发现
- ✓ 测试：使用 `grpcurl` 命令行工具测试 RPC 接口

5. 数据库操作

5.1. 模型生成

go-zero 支持从 DDL 或现有数据库生成 Model 代码，提供开箱即用的 CRUD 操作。

从 DDL 生成：

```

-- user.sql
CREATE TABLE `user` (

```

```

`id` bigint NOT NULL AUTO_INCREMENT,
`name` varchar(255) NOT NULL COMMENT '用户名',
`email` varchar(255) NOT NULL COMMENT '邮箱',
`password` varchar(255) NOT NULL COMMENT '密码',
`status` tinyint NOT NULL DEFAULT '1' COMMENT '状态 1-正常 0-禁用',
`created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
`updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
`deleted_at` timestamp NULL DEFAULT NULL COMMENT '软删除',
PRIMARY KEY (`id`),
UNIQUE KEY `idx_email` (`email`),
KEY `idx_status` (`status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

```

```

# 从 DDL 文件生成 Model
goctl model mysql ddl --src user.sql --dir ./model

# 生成带缓存的 Model(推荐)
goctl model mysql ddl --src user.sql --dir ./model -c

# 指定样式(style: go_zero/go_zero_ddl)
goctl model mysql ddl --src user.sql --dir ./model --style go_zero

```

从数据库生成：

```

# 从数据库表生成
goctl model mysql datasource \
  --url root:password@tcp(localhost:3306)/dbname \
  --table user \
  --dir ./model

# 生成所有表
goctl model mysql datasource \
  --url root:password@tcp(localhost:3306)/dbname \
  --dir ./model

# 生成带缓存的 Model
goctl model mysql datasource \
  --url root:password@tcp(localhost:3306)/dbname \
  --table user \
  --dir ./model \
  -c

```

生成的代码结构：

```

model/
├─ usermodel.go          # Model 接口定义

```

```
└─ userModel_gen.go      # 自动生成的 CRUD 方法
└─ vars.go               # 错误定义
```

Model 接口：

```
// model/usermodel.go
type UserModel interface {
    Insert(ctx context.Context, data *User) (sql.Result, error)
    FindOne(ctx context.Context, id int64) (*User, error)
    FindOneByEmail(ctx context.Context, email string) (*User, error)
    Update(ctx context.Context, data *User) error
    Delete(ctx context.Context, id int64) error
}

// 自动生成的方法（带缓存）
type (
    User struct {
        Id          int64
        Name        string
        Email       string
        Password    string
        Status      int64
        CreatedAt   time.Time
        UpdatedAt   time.Time
        DeletedAt   sql.NullTime
    }
)
```

5.2. CRUD 操作

插入数据：

```
// 插入单条
user := &model.User{
    Name:      "张三",
    Email:     "zhangsan@example.com",
    Password:  "hashed_password",
    Status:    1,
}

result, err := l.svcCtx.UserModel.Insert(l.ctx, user)
if err != nil {
    return nil, err
}

userId, _ := result.LastInsertId()
logx.Infof("插入成功, 用户ID: %d", userId)
```


查询单条：

```
// 根据主键查询
user, err := l.svcCtx.UserModel.FindOne(l.ctx, userId)
if err != nil {
    if err == model.ErrNotFound {
        return nil, errors.New("用户不存在")
    }
    return nil, err
}

// 根据唯一索引查询
user, err := l.svcCtx.UserModel.FindOneByEmail(l.ctx, email)
if err != nil {
    return nil, err
}
```

更新数据：

```
// 更新用户信息
user.Name = "李四"
user.Email = "lisi@example.com"

err := l.svcCtx.UserModel.Update(l.ctx, user)
if err != nil {
    return nil, err
}
```

删除数据：

```
// 物理删除
err := l.svcCtx.UserModel.Delete(l.ctx, userId)

// 软删除（需要表有 deleted_at 字段）
user.DeletedAt = sql.NullTime{
    Time: time.Now(),
    Valid: true,
}
err := l.svcCtx.UserModel.Update(l.ctx, user)
```

批量插入：

```
// 自定义批量插入
func (m *customUserModel) BatchInsert(ctx context.Context, users []*User)
error {
    query := fmt.Sprintf("INSERT INTO %s (name, email, password) VALUES ",
```

```

m.table)

var values []interface{}
var placeholders []string

for _, user := range users {
    placeholders = append(placeholders, "(?, ?, ?)")
    values = append(values, user.Name, user.Email, user.Password)
}

query += strings.Join(placeholders, ",")
_, err := m.conn.ExecCtx(ctx, query, values...)
return err
}

```

5.3. 高级查询

条件查询：

```

// 自定义查询方法（在 userModel.go 中添加）
type UserModel interface {
    // ... 自动生成的方法

    // 自定义方法
    FindByStatus(ctx context.Context, status int64) ([]*User, error)
    FindByEmailLike(ctx context.Context, email string) ([]*User, error)
    Count(ctx context.Context) (int64, error)
}

// 实现
func (m *customUserModel) FindByStatus(ctx context.Context, status int64) ([]*User, error) {
    query := fmt.Sprintf("SELECT %s FROM %s WHERE `status` = ?", userRows, m.table)

    var users []*User
    err := m.conn.QueryRowsCtx(ctx, &users, query, status)
    return users, err
}

func (m *customUserModel) FindByEmailLike(ctx context.Context, email string) ([]*User, error) {
    query := fmt.Sprintf("SELECT %s FROM %s WHERE `email` LIKE ?", userRows, m.table)

    var users []*User
    err := m.conn.QueryRowsCtx(ctx, &users, query, email+"%")
    return users, err
}

```

分页查询：

```
// API 定义
type ListUsersRequest {
    Page      int `form:"page,default=1"`
    PageSize  int `form:"pageSize,default=10,range=[1,100]"`
    Status     int `form:"status,optional"`
    Keyword    string `form:"keyword,optional"`
}

type ListUsersResponse {
    Users []*UserResponse `json:"users"`
    Total int64           `json:"total"`
}

// Logic 实现
func (l *ListUsersLogic) ListUsers(req *types.ListUsersRequest)
(*types.ListUsersResponse, error) {
    offset := (req.Page - 1) * req.PageSize
    limit  := req.PageSize

    // 构建查询条件
    where := "1=1"
    var args []interface{}

    if req.Status > 0 {
        where += " AND status = ?"
        args = append(args, req.Status)
    }

    if req.Keyword != "" {
        where += " AND (name LIKE ? OR email LIKE ?)"
        args = append(args, "%"+req.Keyword+"%", "%"+req.Keyword+"%")
    }

    // 查询总数
    countQuery := fmt.Sprintf("SELECT COUNT(*) FROM user WHERE %s", where)
    var total int64
    err := l.svcCtx.UserModel.QueryRowCtx(l.ctx, &total, countQuery,
args...)

    // 查询列表
    query := fmt.Sprintf("SELECT %s FROM user WHERE %s ORDER BY created_at
DESC LIMIT ?, ?", userRows, where)
    args = append(args, offset, limit)

    var users []*model.User
    err = l.svcCtx.UserModel.QueryRowsCtx(l.ctx, &users, query, args...)

    return &types.ListUsersResponse{
        Users: users,
        Total: total,
    }
}
```

```
    }, nil
}
```

QueryBuilder 模式：

```
// 使用 sqlx 的 QueryBuilder
func (m *customUserModel) FindByConditions(ctx context.Context, conditions
map[string]interface{}) ([]*User, error) {
    builder := strings.Builder{}
    builder.WriteString(fmt.Sprintf("SELECT %s FROM %s WHERE 1=1",
userRows, m.table))

    var args []interface{}

    if name, ok := conditions["name"]; ok {
        builder.WriteString(" AND name LIKE ?")
        args = append(args, "%" + name.(string) + "%")
    }

    if status, ok := conditions["status"]; ok {
        builder.WriteString(" AND status = ?")
        args = append(args, status)
    }

    var users []*User
    err := m.conn.QueryRowsCtx(ctx, &users, builder.String(), args...)
    return users, err
}
```

5.4. 事务处理

使用事务：

```
// 在 ServiceContext 中添加事务管理器
type ServiceContext struct {
    Config config.Config
    DB      *sqlx.DB
}

// 开启事务
func (s *ServiceContext) Transact(fn func(session sqlx.Session) error)
error {
    tx, err := s.DB.Begin()
    if err != nil {
        return err
    }

    defer func() {
        if p := recover(); p != nil {

```

```

        tx.Rollback()
        panic(p)
    }
}()

if err := fn(tx); err != nil {
    tx.Rollback()
    return err
}

return tx.Commit()
}

```

事务示例：

```

// 创建订单（涉及多个表操作）
func (l *CreateOrderLogic) CreateOrder(req *types.CreateOrderRequest) error {
    return l.svcCtx.Transact(func(session sqlx.Session) error {
        // 1. 扣减库存
        product, err := l.svcCtx.ProductModel.FindOne(l.ctx, req.ProductId)
        if err != nil {
            return err
        }

        if product.Stock < req.Quantity {
            return errors.New("库存不足")
        }

        // 更新库存
        updateSQL := "UPDATE product SET stock = stock - ? WHERE id = ?"
        _, err = session.ExecCtx(l.ctx, updateSQL, req.Quantity,
req.ProductId)
        if err != nil {
            return err
        }

        // 2. 创建订单
        order := &model.Order{
            UserId:    req.UserId,
            ProductId: req.ProductId,
            Quantity:  req.Quantity,
            Amount:    product.Price * int64(req.Quantity),
            Status:    0, // 待支付
        }

        result, err := l.svcCtx.OrderModel.Insert(l.ctx, order)
        if err != nil {
            return err
        }

        orderId, _ := result.LastInsertId()
    })
}

```

```

        // 3. 扣减用户余额
        decrBalanceSQL := "UPDATE user SET balance = balance - ? WHERE id =
?"
        _, err = session.ExecCtx(l.ctx, decrBalanceSQL, order.Amount,
req.UserId)
        if err != nil {
            return err
        }

        logx.Infof("订单创建成功: %d", orderId)
        return nil
    })
}

```

分布式事务 (Saga 模式) :

```

// 补偿事务模式
func (l *CreateOrderLogic) CreateOrderSaga(req *types.CreateOrderRequest)
error {
    // 1. 本地创建订单
    order := &model.Order{
        Status: OrderStatusPending,
    }
    _, err := l.svcCtx.OrderModel.Insert(l.ctx, order)
    if err != nil {
        return err
    }

    // 2. 调用库存服务
    stockResp, err := l.svcCtx.StockRpc.DecrStock(l.ctx,
&stock.DecrStockRequest{
        ProductId: req.ProductId,
        Quantity:  req.Quantity,
    })

    if err != nil || !stockResp.Success {
        // 补偿: 取消订单
        l.svcCtx.OrderModel.UpdateStatus(l.ctx, order.Id,
OrderStatusCancelled)
        return errors.New("库存扣减失败")
    }

    // 3. 调用支付服务
    payResp, err := l.svcCtx.PayRpc.Pay(l.ctx, &pay.PayRequest{
        OrderId: order.Id,
        Amount:  order.Amount,
    })

    if err != nil || !payResp.Success {
        // 补偿: 恢复库存
        l.svcCtx.StockRpc.IncrStock(l.ctx, &stock.IncrStockRequest{

```

```
        ProductId: req.ProductId,
        Quantity:  req.Quantity,
    })
    // 补偿：取消订单
    l.svcCtx.OrderModel.UpdateStatus(l.ctx, order.Id,
    OrderStatusCancelled)
    return errors.New("支付失败")
}

// 4. 更新订单状态
l.svcCtx.OrderModel.UpdateStatus(l.ctx, order.Id, OrderStatusPaid)

return nil
}
```

小节：

- ✓ 模型生成：从 DDL 或数据库生成，支持缓存
- ✓ CRUD 操作：自动生成基本操作，支持自定义方法
- ✓ 高级查询：条件查询、分页、QueryBuilder
- ✓ 事务处理：本地事务和分布式事务
- ✓ 最佳实践：使用缓存、批量操作、事务保证一致性

6. 配置与日志

6.1. 配置文件结构

YAML 配置文件：

```
# etc/myapi-api.yaml
Name: myapi-api      # 服务名称
Host: 0.0.0.0         # 监听地址
Port: 8888            # 监听端口

# 日志配置
Log:
  ServiceName: myapi-api
  Mode: console      # console/file/stdout
  Level: info        # debug/info/error/severe
  Path: logs         # 日志目录
  KeepDays: 7        # 保留天数
  Compress: true     # 压缩日志

# 数据库配置
Mysql:
  DataSource: root:password@tcp(localhost:3306)/dbname?
charset=utf8mb4&parseTime=true&loc=Local
  MaxOpenConns: 100   # 最大连接数
  MaxIdleConns: 10    # 最大空闲连接
  ConnMaxLifetime: 3600 # 连接最大存活时间(秒)
```

```

# Redis 配置
Redis:
  Host: localhost:6379
  Type: node           # node/cluster
  Password: ""
  DB: 0
  # 集群模式
  # Addrs: [localhost:7001, localhost:7002]

# RPC 服务配置
UserRpc:
  Etcd:
    Hosts:
      - localhost:2379
    Key: user.rpc
  Timeout: 3000        # 超时时间(毫秒)

# 监控配置
Prometheus:
  Host: 0.0.0.0
  Port: 9091
  Path: /metrics

# 链路追踪
Telemetry:
  Name: myapi-api
  Endpoint: http://localhost:14268/api/traces
  Sampler: 1.0         # 采样率 0-1
  Batcher: jaeger      # jaeger/zipkin

```

配置结构体：

```

// internal/config/config.go
type Config struct {
  rest.RestConf // 内置 REST 配置

  // 数据库
  Mysql struct {
    DataSource string
    MaxOpenConns int `json:",default=100"`
    MaxIdleConns int `json:",default=10"`
  }

  // Redis
  Redis redis.RedisConf

  // RPC 服务
  UserRpc zrpc.RpcClientConf
  OrderRpc zrpc.RpcClientConf

  // 自定义配置

```



```

    JwtSecret string `json:",env=JWT_SECRET"` // 支持环境变量
    TokenExpire int `json:",default=86400"`    // 默认值
}

```

配置加载：

```

// myapi.go
var configFile = flag.String("f", "etc/myapi-api.yaml", "配置文件路径")

func main() {
    flag.Parse()

    // 加载配置
    var c config.Config
    conf.MustLoad(*configFile, &c)

    // 创建服务
    server := rest.MustNewServer(c.RestConf)
    defer server.Stop()

    // 初始化上下文
    ctx := svc.NewServiceContext(c)
    handler.RegisterHandlers(server, ctx)

    // 启动服务
    server.Start()
}

```

6.2. 日志系统

基本使用：

```

import "github.com/zeromicro/go-zero/core/logx"

// 不同级别的日志
logx.Debug("调试信息")    // 仅在开发环境输出
logx.Info("普通信息")     // 生产环境默认级别
logx.Warn("警告信息")     // 警告级别
logx.Error("错误信息")    // 错误级别
logx.Severe("严重错误")   // 严重级别

// 格式化日志
logx.Infof("用户 %s 登录成功", username)
logx.Errorf("查询失败: %v", err)

// 结构化日志（推荐）
logx.Infow("用户登录",
    logx.Field("userId", userId),
    logx.Field("username", username),

```

```
        logx.Field("ip", ip),
    )

    // 错误日志
    logx.Errorw("数据库错误",
        logx.Field("error", err.Error()),
        logx.Field("sql", sql),
    )
}
```

在 Logic 中使用日志：

```
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    // 记录请求开始
    logx.Infow("开始查询用户", logx.Field("userId", req.Id))

    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        logx.Errorw("查询用户失败",
            logx.Field("userId", req.Id),
            logx.Field("error", err.Error()),
        )
        return nil, err
    }

    // 记录成功
    logx.Infow("查询用户成功", logx.Field("userId", req.Id))

    return &types.UserResponse{
        Id:      user.Id,
        Name:    user.Name,
        Email:   user.Email,
    }, nil
}
```

日志配置详解：

配置项	说明	推荐值
Mode	输出模式：console/file/stdout	开发：console，生产：file
Level	日志级别：debug/info/error/severe	开发：debug，生产：info
Path	日志文件目录	/var/log/app
KeepDays	保留天数	7-30 天
Compress	是否压缩	true
MaxSize	单文件最大大小(MB)	100

日志输出格式：

```
# JSON 格式 (推荐生产环境)
{
  "level": "info",
  "ts": "2024-01-15T10:30:00.123+0800",
  "caller": "logic/get_user_logic.go:25",
  "content": "用户登录",
  "userId": 123,
  "username": "张三",
  "ip": "192.168.1.100"
}

# 文本格式 (开发环境)
2024/01/15 10:30:00 [info] logic/get_user_logic.go:25 用户登录 userId=123
username=张三 ip=192.168.1.100
```

6.3. 多环境管理

多环境配置文件：

```
etc/
├─ myapi-api.yaml           # 默认配置
├─ myapi-api-dev.yaml      # 开发环境
├─ myapi-api-test.yaml     # 测试环境
└─ myapi-api-prod.yaml     # 生产环境
```

不同环境配置：

```
# myapi-api-dev.yaml
Name: myapi-api
Host: 0.0.0.0
Port: 8888

Log:
  Mode: console
  Level: debug

Mysql:
  DataSource: root:password@tcp(localhost:3306)/dbname_dev

# myapi-api-prod.yaml
Name: myapi-api
Host: 0.0.0.0
Port: 80

Log:
  Mode: file
  Level: info
  Path: /var/log/myapi
```

```
Mysql:
  DataSource: ${DB_USER}:${DB_PASSWORD}@tcp(${DB_HOST}:3306)/${DB_NAME}
```

环境变量支持：

```
# 支持环境变量
Mysql:
  DataSource: ${DB_USER}:${DB_PASSWORD}@tcp(${DB_HOST}:3306)/${DB_NAME}

# 在配置结构体中使用环境变量
type Config struct {
  JwtSecret string `json:",env=JWT_SECRET"`
  DbPassword string `json:",env=DB_PASSWORD"`
}
```

配置加密：

```
# 生成加密密钥
goctl env secret -plaintext "your-password"
# 输出: @encrypted(xxxxx)

# 在配置文件中使用
Mysql:
  DataSource: root:@encrypted(xxxxx)@tcp(localhost:3306)/dbname
```

启动时指定配置：

```
# 开发环境
go run myapi.go -f etc/myapi-api-dev.yaml

# 生产环境
./myapi -f etc/myapi-api-prod.yaml

# 使用环境变量
export CONFIG_FILE=etc/myapi-api-prod.yaml
./myapi -f $CONFIG_FILE
```

小节：

- ✓ 配置结构：YAML 格式，支持默认值、环境变量
- ✓ 日志系统：多级别、结构化、可配置输出方式
- ✓ 多环境管理：使用不同配置文件，支持环境变量和加密
- ✓ 最佳实践：开发用 console，生产用 file，敏感信息加密

7. 缓存系统

7.1. Redis 配置

单节点配置：

```
# etc/myapi-api.yaml
Redis:
  Host: localhost:6379
  Type: node           # 单节点
  Password: ""         # 密码
  DB: 0                # 数据库索引
  MaxIdle: 100          # 最大空闲连接
  MaxActive: 200        # 最大活跃连接
  IdleTimeout: 300      # 空闲超时(秒)
```

集群配置：

```
Redis:
  Type: cluster        # 集群模式
  Addrs:
    - localhost:7001
    - localhost:7002
    - localhost:7003
  Password: ""
  MaxIdle: 100
  MaxActive: 200
```

哨兵配置：

```
Redis:
  Type: sentinel       # 哨兵模式
  MasterName: mymaster
  Addrs:
    - localhost:26379
    - localhost:26380
    - localhost:26381
  Password: ""
```

初始化 Redis：

```
// internal/svc/servicecontext.go
func NewServiceContext(c config.Config) *ServiceContext {
    return &ServiceContext{
        Config: c,
        Redis:  redis.New(c.Redis.Host), // 单节点
    }
}
```

```

        // 或使用配置
        // Redis: redis.MustNewRedis(c.Redis),
    }
}

```

7.2. 缓存策略

手动缓存：

```

// 查询缓存
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    cacheKey := fmt.Sprintf("user:%d", req.Id)

    // 1. 先查缓存
    cached, err := l.svcCtx.Redis.Get(cacheKey)
    if err == nil && cached != "" {
        var user types.UserResponse
        json.Unmarshal([]byte(cached), &user)
        return &user, nil
    }

    // 2. 查询数据库
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        return nil, err
    }

    // 3. 写入缓存
    data, _ := json.Marshal(user)
    l.svcCtx.Redis.Setex(cacheKey, string(data), 3600) // 1小时过期

    return user, nil
}

```

防缓存穿透：

```

import "github.com/zeromicro/go-zero/core/stores/cache"

// 使用 cache.Take 自动处理缓存
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    var user types.UserResponse
    cacheKey := fmt.Sprintf("user:%d", req.Id)

    err := cache.Take(&user, cacheKey, func() (interface{}, error) {
        // 缓存不存在时调用
        u, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
        if err != nil {

```

```
        return nil, err
    }

    return types.UserResponse{
        Id:    u.Id,
        Name:  u.Name,
        Email: u.Email,
    }, nil
})

return &user, err
}
```

缓存更新策略：

策略	说明	适用场景	示例
Cache-Aside	先更新 DB，再删除缓存	读多写少	推荐
Read-Through	缓存穿透读 DB	读密集	cache.Take
Write-Through	先写缓存，再写 DB	写密集	双写
Write-Behind	先写缓存，异步写 DB	高性能	队列异步

Cache-Aside 模式实现：

```
// 更新用户
func (l *UpdateUserLogic) UpdateUser(req *types.UpdateUserRequest) error {
    // 1. 更新数据库
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        return err
    }

    user.Name = req.Name
    user.Email = req.Email

    if err := l.svcCtx.UserModel.Update(l.ctx, user); err != nil {
        return err
    }

    // 2. 删除缓存（下次查询时重新加载）
    cacheKey := fmt.Sprintf("user:%d", req.Id)
    _, err = l.svcCtx.Redis.Del(cacheKey)

    return err
}
```

缓存穿透解决方案：

```

// 1. 缓存空值
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    cacheKey := fmt.Sprintf("user:%d", req.Id)

    // 查询缓存
    cached, _ := l.svcCtx.Redis.Get(cacheKey)
    if cached == "null" {
        // 空值缓存, 直接返回
        return nil, errors.New("用户不存在")
    }

    // 查询数据库
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err == model.ErrNotFound {
        // 缓存空值, 防止穿透
        l.svcCtx.Redis.Setex(cacheKey, "null", 60) // 60秒过期
        return nil, errors.New("用户不存在")
    }

    // ... 其他逻辑
}

// 2. 布隆过滤器
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    // 先检查布隆过滤器
    if !l.svcCtx.BloomFilter.Test([]byte(fmt.Sprintf("%d", req.Id))) {
        return nil, errors.New("用户不存在")
    }

    // 再查缓存和数据库
    // ...
}

```

7.3. 缓存最佳实践

缓存 Key 设计：

格式：业务:模块:唯一标识

示例：

- 用户信息：user:info:123
- 用户订单：user:order:123:456
- 商品库存：product:stock:789
- 会话信息：session:token:abc123

过期时间设置：

数据类型	过期时间	说明
用户信息	1-2 小时	变更较少
商品信息	30-60 分钟	中等变更
库存信息	5-10 分钟	频繁变更
验证码	5-10 分钟	短期有效
Session	7-30 天	长期有效

缓存预热：

```
// 程序启动时预热热点数据
func (s *ServiceContext) Warmup() error {
    // 预加载热点商品
    hotProducts, _ := s.ProductModel.FindHotProducts(context.Background(),
100)

    for _, product := range hotProducts {
        key := fmt.Sprintf("product:info:%d", product.Id)
        data, _ := json.Marshal(product)
        s.Redis.Setex(key, string(data), 3600)
    }

    logx.Info("缓存预热完成")
    return nil
}
```

缓存雪崩防护：

```
// 过期时间加随机值，避免同时失效
func setCache(redis *redis.Redis, key string, value string, baseExpire int)
{
    // 基础过期时间 + 随机值(0-300秒)
    expire := baseExpire + rand.Intn(300)
    redis.Setex(key, value, expire)
}
```

缓存击穿防护：

```
// 使用分布式锁防止并发穿透
func (l *GetUserLogic) GetUser(req *types.GetUserRequest)
(*types.UserResponse, error) {
    cacheKey := fmt.Sprintf("user:%d", req.Id)

    // 查询缓存
    cached, _ := l.svcCtx.Redis.Get(cacheKey)
```

```
if cached != "" {
    var user types.UserResponse
    json.Unmarshal([]byte(cached), &user)
    return &user, nil
}

// 获取分布式锁
lockKey := fmt.Sprintf("lock:%s", cacheKey)
lock, err := redis.Lock(l.svcCtx.Redis, lockKey, 10*time.Second)
if err != nil {
    // 获取锁失败，等待并重试
    time.Sleep(100 * time.Millisecond)
    return l.GetUser(req) // 递归重试
}
defer lock.Unlock()

// 双重检查（防止重复查询）
cached, _ = l.svcCtx.Redis.Get(cacheKey)
if cached != "" {
    var user types.UserResponse
    json.Unmarshal([]byte(cached), &user)
    return &user, nil
}

// 查询数据库并写入缓存
user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
if err != nil {
    return nil, err
}

data, _ := json.Marshal(user)
l.svcCtx.Redis.Setex(cacheKey, string(data), 3600)

return user, nil
}
```

小节：

- ✓ Redis 配置：支持单节点、集群、哨兵模式
- ✓ 缓存策略：Cache-Aside、Read-Through、Write-Through
- ✓ 防穿透：缓存空值、布隆过滤器
- ✓ 防雪崩：过期时间加随机值
- ✓ 防击穿：分布式锁
- ✓ 最佳实践：合理的 Key 设计和过期时间

8. 中间件与拦截器

8.1. 自定义中间件

日志中间件：

```
// internal/middleware/logging_middleware.go
package middleware

import (
    "net/http"
    "time"

    "github.com/zeromicro/go-zero/core/logx"
)

func LoggingMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        // 记录请求信息
        logx.Infow("请求开始",
            logx.Field("method", r.Method),
            logx.Field("path", r.URL.Path),
            logx.Field("ip", r.RemoteAddr),
        )

        // 调用下一个处理器
        next(w, r)

        // 记录响应时间
        logx.Infow("请求结束",
            logx.Field("duration", time.Since(start).String()),
        )
    }
}
```

认证中间件：

```
// internal/middleware/auth_middleware.go
func AuthMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        // 从 Header 获取 token
        token := r.Header.Get("Authorization")
        if token == "" {
            http.Error(w, "未授权", http.StatusUnauthorized)
            return
        }

        // 验证 token
        userId, err := validateToken(token)
        if err != nil {
            http.Error(w, "token 无效", http.StatusUnauthorized)
            return
        }

        // 将用户信息放入 Context
    }
}
```

```

        ctx := context.WithValue(r.Context(), "userId", userId)
        next(w, r.WithContext(ctx))
    }
}

```

限流中间件：

```

// internal/middleware/rate_limit_middleware.go
func RateLimitMiddleware(limit int) func(http.HandlerFunc) http.HandlerFunc {
    limiter := rate.NewLimiter(rate.Limit(limit), limit*2)

    return func(next http.HandlerFunc) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
            if !limiter.Allow() {
                http.Error(w, "请求过于频繁", http.StatusTooManyRequests)
                return
            }
            next(w, r)
        }
    }
}

```

注册中间件：

```

// 在 API 定义中使用
@server(
    middleware: LoggingMiddleware, AuthMiddleware
)
service my-api {
    @handler getUser
    get /user/:id (GetUserRequest) returns (UserResponse)
}

// 或者在代码中手动注册
func main() {
    server := rest.MustNewServer(c.RestConf)
    defer server.Stop()

    // 全局中间件
    server.Use(middleware.LoggingMiddleware)
    server.Use(middleware.RecoverMiddleware)

    // 路由级中间件
    server.AddRoutes(
        []rest.Route{
            {
                Method: "GET",
                Path: "/user/:id",
            }
        }
    )
}

```

```

        Handler:
        middleware.AuthMiddleware(handler.GetUserHandler(ctx)),
    },
},
)
}

```

8.2. 内置中间件

Recover 中间件：

```

import "github.com/zeromicro/go-zero/rest/handler"

// 异常恢复（必须使用，防止 panic）
server.Use(handler.RecoverHandler)

```

CORS 跨域中间件：

```

import "github.com/zeromicro/go-zero/rest/handler"

// CORS 配置
server.Use(handler.NewCorsHandler(handler.CorsConf{
    AllowedOrigins: []string{"*"}, // 允许的域名
    AllowedMethods: []string{"GET", "POST", "PUT", "DELETE"},
    AllowedHeaders: []string{"Content-Type", "Authorization"},
    ExposedHeaders: []string{},
    AllowCredentials: true,
    MaxAge:          3600,
}))

```

请求超时中间件：

```

func TimeoutMiddleware(timeout time.Duration) func(http.HandlerFunc)
http.HandlerFunc {
    return func(next http.HandlerFunc) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
            ctx, cancel := context.WithTimeout(r.Context(), timeout)
            defer cancel()

            // 使用带超时的 Context
            r = r.WithContext(ctx)

            done := make(chan bool)
            go func() {
                next(w, r)
                done <- true
            }()
        }()
    }
}

```

```

        select {
        case <-done:
            return
        case <-ctx.Done():
            http.Error(w, "请求超时", http.StatusRequestTimeout)
            return
        }
    }
}

```

8.3. 认证与鉴权

JWT 认证实现：

```

// internal/auth/jwt.go
package auth

import (
    "errors"
    "time"

    "github.com/golang-jwt/jwt/v4"
)

type JwtClaims struct {
    UserId    int64 `json:"userId"`
    Username  string `json:"username"`
    jwt.RegisteredClaims
}

var JwtSecret = []byte("your-secret-key")

// 生成 Token
func GenerateToken(userId int64, username string) (string, error) {
    claims := JwtClaims{
        UserId:    userId,
        Username:  username,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
            Issuer:    "myapi",
        },
    },
}

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(JwtSecret)
}

// 解析 Token

```

```

func ParseToken(tokenString string) (*JwtClaims, error) {
    token, err := jwt.ParseWithClaims(tokenString, &JwtClaims{}, func(token
*jwt.Token) (interface{}, error) {
        return JwtSecret, nil
    })

    if err != nil {
        return nil, err
    }

    if claims, ok := token.Claims.(*JwtClaims); ok && token.Valid {
        return claims, nil
    }

    return nil, errors.New("invalid token")
}

// JWT 中间件
func JwtMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        authHeader := r.Header.Get("Authorization")
        if authHeader == "" {
            http.Error(w, "未授权", http.StatusUnauthorized)
            return
        }

        // Bearer token
        tokenString := strings.TrimPrefix(authHeader, "Bearer ")
        claims, err := ParseToken(tokenString)
        if err != nil {
            http.Error(w, "token 无效", http.StatusUnauthorized)
            return
        }

        // 将用户信息放入 Context
        ctx := context.WithValue(r.Context(), "claims", claims)
        next(w, r.WithContext(ctx))
    }
}

```

在 API 中使用 JWT :

```

// user.api
@server(
    prefix: /api/v1
    jwt: Auth // 启用 JWT 认证
)
service user-api {
    @handler getProfile
    get /user/profile returns (UserResponse)
}

```

```
// 在 Logic 中获取用户信息
func (l *GetProfileLogic) GetProfile() (*types.UserResponse, error) {
    // 从 Context 获取用户 ID
    userId, _ := l.ctx.Value("userId").(json.Number).Int64()

    user, err := l.svcCtx.UserModel.FindOne(l.ctx, userId)
    if err != nil {
        return nil, err
    }

    return &types.UserResponse{
        Id:      user.Id,
        Name:    user.Name,
        Email:   user.Email,
    }, nil
}
```

RBAC 权限控制：

```
// internal/middleware/rbac_middleware.go
func RBACMiddleware(permissions map[string][]string) func(http.HandlerFunc)
http.HandlerFunc {
    return func(next http.HandlerFunc) http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
            // 从 Context 获取用户角色
            claims, ok := r.Context().Value("claims").(*auth.JwtClaims)
            if !ok {
                http.Error(w, "未授权", http.StatusUnauthorized)
                return
            }

            // 获取用户角色
            role := getUserRole(claims.UserId)

            // 检查权限
            path := r.URL.Path
            if requiredPerms, exists := permissions[path]; exists {
                if !hasPermission(role, requiredPerms) {
                    http.Error(w, "权限不足", http.StatusForbidden)
                    return
                }
            }

            next(w, r)
        }
    }
}
```

8.4. 统一响应处理

统一响应格式：

```
// internal/types/response.go
package types

type Response struct {
    Code    int        `json:"code"`
    Message string     `json:"message"`
    Data    interface{} `json:"data,omitempty"`
    TraceId string     `json:"traceId,omitempty"`
}

type PageResponse struct {
    Code    int        `json:"code"`
    Message string     `json:"message"`
    Data    interface{} `json:"data,omitempty"`
    Total   int64      `json:"total"`
    Page    int        `json:"page"`
    PageSize int       `json:"pageSize"`
}

// 成功响应
func Success(data interface{}) Response {
    return Response{
        Code:    0,
        Message: "success",
        Data:    data,
    }
}

// 失败响应
func Error(code int, message string) Response {
    return Response{
        Code:    code,
        Message: message,
    }
}

// 分页响应
func PageSuccess(data interface{}, total int64, page, pageSize int)
PageResponse {
    return PageResponse{
        Code:    0,
        Message: "success",
        Data:    data,
        Total:   total,
        Page:    page,
        PageSize: pageSize,
    }
}
```

响应中间件：

```
// 统一错误处理中间件
func ErrorHandlerMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                logx.ErrorStack(err)
                httpx.OkJson(w, types.Error(500, "服务器内部错误"))
            }
        }()

        next(w, r)
    }
}
```

使用示例：

```
func (l *GetUserLogic) GetUser(req *types.GetUserRequest) (*types.Response,
error) {
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        if err == model.ErrNotFound {
            return nil, errors.New("用户不存在")
        }
        return nil, err
    }

    return types.Success(&types.UserResponse{
        Id:    user.Id,
        Name:  user.Name,
        Email: user.Email,
    }), nil
}
```

小节：

- ✓ 自定义中间件：日志、认证、限流等
- ✓ 内置中间件：Recover、CORS、超时控制
- ✓ JWT 认证：生成、解析、验证 Token
- ✓ RBAC 权限：基于角色的访问控制
- ✓ 统一响应：标准化的响应格式

9. 服务治理

9.1. 超时与重试

为什么需要超时控制：

- 防止请求无限等待，资源耗尽
- 保护下游服务，避免雪崩效应

```
Timeout:
  Default: 5000      # 默认超时 5s (推荐)
  Long: 10000       # 长任务超时 10s
```

```
// RPC 客户端超时配置
clientConf := &zrpc.RpcClientConf{
  Timeout:    3000,      // 请求超时 3s
  RetryCalls: true,      // 启用重试
  MaxRetries: 3,         // 最大重试次数
}
```

重试策略：

策略	说明
快速失败	立即返回错误
固定间隔	每次重试间隔固定时间
指数退避	重试间隔递增（推荐）

9.2. 限流

限流算法：

算法	说明	适用场景
令牌桶	允许突发流量	API 限流
漏桶	均匀处理请求	流量整形
计数器	简单粗暴	简单限流

限流配置：

```
import "github.com/zeromicro/go-zero/core/limit"

// 令牌桶限流（推荐）
engine.Use(ratelimit.New(
  ratelimit.WithMaxEvents(100), // 每秒令牌数
  ratelimit.WithBurst(200),    // 突发容量
))

// 按接口限流
engine.AddRoutes([]rest.Route{
  {
```

```

        Method: "GET",
        Path:    "/api/user",
        Handler: ratelimitHandler(
            ratelimit.WithMaxEvents(50),
        )(userHandler),
    },
})

```

常见限流场景：

```

// 接口级限流
engine.Use(ratelimit.New(ratelimit.WithMaxEvents(100)))

// IP 限流（自定义实现）
func IPLimitMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        ip := r.RemoteAddr
        key := fmt.Sprintf("ratelimit:%s", ip)

        // 使用 Redis 实现分布式限流
        count, _ := redis.Incr(key)
        if count == 1 {
            redis.Expire(key, time.Second)
        }
        if count > 100 {
            http.Error(w, "Too Many Requests", http.StatusTooManyRequests)
            return
        }
        next(w, r)
    }
}

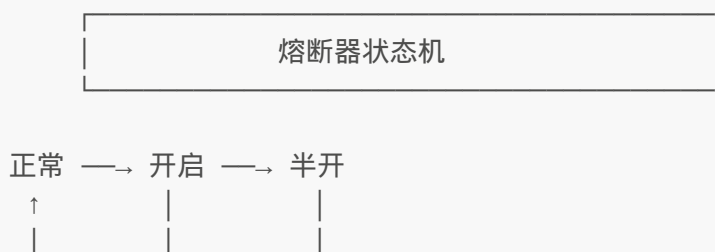
```

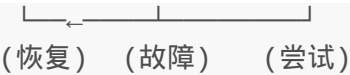
9.3. 熔断

什么是熔断：

- 当下游服务故障时，快速失败返回
- 防止故障蔓延，保护系统可用性
- 自动检测恢复，自动闭合

熔断器三种状态：





- Closed (闭合)：正常状态，请求通过
- Open (开启)：故障状态，直接拒绝
- Half-Open (半开)：尝试状态，允许部分请求

熔断配置：

```
import "github.com/zeromicro/go-zero/core/breaker"

// 全局熔断器
brk := breaker.GetBreaker("user-service")

// 手动熔断
brk.MarkFailure() // 记录失败
brk.MarkSuccess() // 记录成功

// 自动熔断调用
err := brk.Do(func() error {
    return callRemoteService()
})
// 失败返回 ErrBreakerOpen
```

熔断参数配置：

```
// 方式1：全局默认配置
breaker.SetConfig(breaker.Config{
    RequestVolume: 10, // 统计请求数窗口
    SleepThreshold: 3, // 熔断后等待时间(秒)
    Threshold: 50, // 失败率阈值(%)
})

// 方式2：服务级别配置
clientConf := &zrpc.RpcClientConf{
    Breaker: "custom-breaker",
}
```

9.4. 降级

降级策略：

策略	说明	示例
返回默认值	返回预设值	商品详情 → 返回"商品已下架"
读取缓存	用缓存数据	数据库故障 → 读缓存

策略	说明	示例
熔断返回	直接拒绝	快速失败
兜底接口	调用备用服务	主服务故障→备用服务

降级实现：

```
func (l *GetProductLogic) GetProduct(req *ProductRequest) (*Product, error) {
    // 主逻辑：查询数据库
    product, err := l.svcCtx.ProductModel.FindOne(l.ctx, req.Id)
    if err != nil {
        // 降级策略1：读取缓存
        cached, err := l.svcCtx.Redis.Get(fmt.Sprintf("product:%d",
req.Id))
        if err == nil && cached != "" {
            var p Product
            json.Unmarshal([]byte(cached), &p)
            logx.Info("使用缓存数据")
            return &p, nil
        }

        // 降级策略2：返回默认值
        if err == model.ErrNotFound {
            return &Product{
                Name:    "商品已下架",
                Status: 0,
            }, nil
        }

        return nil, err
    }
    return product, nil
}
```

9.5. 负载均衡与健康检查

```
// 负载均衡策略
clientConf := &zrpc.RpcClientConf{
    BalancerName: "random",           // 随机
    // BalancerName: "roundrobin",    // 轮询
    // BalancerName: "leastload",     // 最少负载
}

// 服务注册发现 (Etcd)
Etcd:
    Hosts: [localhost:2379]
    Key: user.rpc
```

10. 错误处理与响应规范

10.1. 错误类型定义

自定义错误类型：

```
// internal/errors/errors.go
package errors

import "fmt"

type CodeError struct {
    Code    int
    Message string
}

func (e *CodeError) Error() string {
    return fmt.Sprintf("code: %d, message: %s", e.Code, e.Message)
}

func New(code int, message string) *CodeError {
    return &CodeError{
        Code:    code,
        Message: message,
    }
}

// 预定义错误
var (
    ErrBadRequest      = New(400, "请求参数错误")
    ErrUnauthorized    = New(401, "未授权")
    ErrForbidden        = New(403, "权限不足")
    ErrNotFound         = New(404, "资源不存在")
    ErrInternal         = New(500, "服务器内部错误")
)
```

业务错误码：

```
// internal/errors/code.go
const (
    // 通用错误码 1000-1999
    CodeSuccess          = 0
    CodeBadRequest        = 1000
    CodeUnauthorized      = 1001
    CodeForbidden         = 1003
    CodeNotFound          = 1004
    CodeInternalServerError = 1005

    // 用户相关错误 2000-2999
)
```

```

    CodeUserNotFound      = 2000
    CodeUserAlreadyExists = 2001
    CodePasswordError     = 2002

    // 订单相关错误 3000-3999
    CodeOrderNotFound     = 3000
    CodeOrderExpired      = 3001
    CodeOrderPaid         = 3002

    // 支付相关错误 4000-4999
    CodePaymentFailed     = 4000
    CodeBalanceNotEnough  = 4001
)

var ErrorMessages = map[int]string{
    CodeSuccess:      "成功",
    CodeBadRequest:   "请求参数错误",
    CodeUserNotFound: "用户不存在",
    CodeOrderNotFound: "订单不存在",
    CodePaymentFailed: "支付失败",
}

```

10.2. 统一响应格式

响应结构：

```

// internal/types/response.go
type Response struct {
    Code      int      `json:"code"`
    Message   string   `json:"message"`
    Data      interface{} `json:"data,omitempty"`
    TraceId   string   `json:"traceId,omitempty"`
}

// 分页响应
type PageResponse struct {
    Code      int      `json:"code"`
    Message   string   `json:"message"`
    Data      interface{} `json:"data,omitempty"`
    Total     int64    `json:"total"`
    Page      int      `json:"page"`
    PageSize  int      `json:"pageSize"`
}

// 构造响应
func Success(data interface{}) Response {
    return Response{
        Code:      0,
        Message:   "success",
        Data:      data,
    }
}

```



```

}

func Error(code int, message string) Response {
    return Response{
        Code:    code,
        Message: message,
    }
}

func PageSuccess(data interface{}, total int64, page, pageSize int)
PageResponse {
    return PageResponse{
        Code:    0,
        Message: "success",
        Data:    data,
        Total:   total,
        Page:    page,
        PageSize: pageSize,
    }
}

```

错误处理中间件：

```

// internal/middleware/error_middleware.go
func ErrorHandlerMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                logx.ErrorStack(err)
                httpx.OkJson(w, Response{
                    Code:    500,
                    Message: "服务器内部错误",
                })
            }
        }()

        next(w, r)
    }
}

```

10.3. 错误处理最佳实践

Logic 层错误处理：

```

func (l *GetUserLogic) GetUser(req *types.GetUserRequest) (*types.Response,
error) {
    // 参数校验
    if req.Id <= 0 {
        return nil, errors.NewBadRequest("用户ID无效")
    }
}

```

```

    }

    // 查询用户
    user, err := l.svcCtx.UserModel.FindOne(l.ctx, req.Id)
    if err != nil {
        if err == model.ErrNotFound {
            return nil, errors.New(errors.CodeUserNotFound, "用户不存在")
        }
        logx.Errorf("查询用户失败: %v", err)
        return nil, errors.NewInternalError("查询失败")
    }

    return types.Success(&types.UserResponse{
        Id:    user.Id,
        Name:  user.Name,
        Email: user.Email,
    }), nil
}

```

Handler 层统一处理：

```

// internal/handler/handler.go
func HandleError(w http.ResponseWriter, err error) {
    if codeErr, ok := err.(*errors.CodeError); ok {
        httpx.OkJson(w, types.Error(codeErr.Code, codeErr.Message))
    } else {
        logx.Error(err)
        httpx.OkJson(w, types.Error(500, "服务器内部错误"))
    }
}

// 使用示例
func GetUserHandler(svcCtx *svc.ServiceContext) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        var req types.GetUserRequest
        if err := httpx.Parse(r, &req); err != nil {
            HandleError(w, errors.NewBadRequest("参数解析失败"))
            return
        }

        l := logic.NewGetUserLogic(r.Context(), svcCtx)
        resp, err := l.GetUser(&req)
        if err != nil {
            HandleError(w, err)
            return
        }

        httpx.OkJson(w, resp)
    }
}

```

小节：

- ✓ 错误类型：自定义错误类型和错误码
- ✓ 响应格式：统一的 JSON 响应结构
- ✓ 错误处理：Logic 层返回错误，Handler 层统一处理
- ✓ 最佳实践：预定义错误码、日志记录、错误追踪

11. 常用功能

11.1. 文件上传下载

```
// 上传
file, header, _ := l.r.FormFile("file")
defer file.Close()
dst, _ := os.Create(filepath.Join("/uploads", header.Filename))
io.Copy(dst, file)

// 下载
l.w.Header().Set("Content-Disposition", "attachment; filename="+filename)
http.ServeFile(l.w, l.r, filepath)
```

11.2. ID 生成器

```
import "github.com/zeromicro/go-zero/core/stores/sqlx"

// 雪花算法（机器 ID 0-1023）
node := sqlx.MustNewSnowflake(1)
id := node.Generate()
```

11.3. 定时任务

```
import "github.com/zeromicro/go-zero/core/task"

// 每分钟执行
task.Schedule("@every 1m", func() { /* ... */ })

// Cron 表达式
task.Schedule("0 0 * * *", func() { /* 每天凌晨 */ })
```

11.4. 分布式锁

为什么需要分布式锁：

- 多实例部署时，保证同一时刻只有一个线程执行关键代码
- 防止并发导致的数据不一致

分布式锁特性：

特性	说明
互斥	同一时间只能一个客户端持有锁
可重入	同一客户端可重复获取锁
失效机制	防止死锁，设置过期时间
高可用	分布式环境下的可靠性

基本使用：

```
import "github.com/zeromicro/go-zero/core/stores/redis"

lock, err := redis.Lock(l.svcCtx.Redis, "lock:order:123", 30*time.Second)
if err != nil {
    return errors.New("系统繁忙，请稍后重试")
}
defer lock.Unlock()
// 业务逻辑...
```

参数说明：

参数	说明	推荐值
key	锁的唯一标识	业务相关前缀+业务ID
expire	锁过期时间	视业务耗时而定
retry	重试次数	2-3次
retryDelay	重试间隔	100-200ms

高级用法：

```
// 带重试的分布式锁
func acquireLockWithRetry(r redis.Redis, key string, expire time.Duration) error {
    for i := 0; i < 3; i++ {
        lock, err := redis.Lock(r, key, expire)
        if err == nil {
            return nil
        }
        time.Sleep(200 * time.Millisecond)
    }
    return errors.New("获取锁失败")
}

// 可重入锁（同一进程内）
var localLocks sync.Map // 存储本地锁计数
```

```
func acquireReentrantLock(r redis.Redis, key string, expire time.Duration,
owner string) error {
    // 检查本地是否已持有锁
    if count, ok := localLocks.Load(owner); ok && count.(int) > 0 {
        localLocks.Store(owner, count.(int)+1)
        return nil
    }

    lock, err := redis.Lock(r, key, expire)
    if err != nil {
        return err
    }
    localLocks.Store(owner, 1)
    return nil
}
```

典型场景：

```
// 场景1：库存扣减
func (l *ReduceStockLogic) ReduceStock(req *ReduceStockRequest) error {
    // 获取锁
    lockKey := fmt.Sprintf("lock:stock:%d", req.ProductId)
    lock, err := redis.Lock(l.svcCtx.Redis, lockKey, 10*time.Second)
    if err != nil {
        return errors.New("系统繁忙")
    }
    defer lock.Unlock()

    // 查询库存
    stock, _ := l.svcCtx.StockModel.FindOneByProductId(l.ctx,
req.ProductId)
    if stock.Num < req.Num {
        return errors.New("库存不足")
    }

    // 扣减库存
    stock.Num -= req.Num
    return l.svcCtx.StockModel.Update(l.ctx, stock)
}

// 场景2：支付订单
func (l *PayOrderLogic) PayOrder(req *PayOrderRequest) error {
    lockKey := fmt.Sprintf("lock:order:%d", req.OrderId)
    lock, err := redis.Lock(l.svcCtx.Redis, lockKey, 30*time.Second)
    if err != nil {
        return errors.New("订单处理中，请稍后重试")
    }
    defer lock.Unlock()

    // 检查订单状态
    order, err := l.svcCtx.OrderModel.FindOne(l.ctx, req.OrderId)
```

```
    if err != nil {
        return err
    }
    if order.Status != 0 {
        return errors.New("订单状态异常")
    }

    // 执行支付
    // ...
    return nil
}
```

注意事项：

- 锁的过期时间要大于业务执行时间
- 尽量缩小锁的范围，减少持锁时间
- 做好异常处理，确保锁能释放

11.5. 事务处理

本地事务：

```
err := l.svcCtx.Trans.Transact(func(session sqlx.Session) error {
    // 多个数据库操作在同一事务中

    // 操作1：创建订单
    _, err := orderModel.Insert(l.ctx, order)
    if err != nil {
        return err
    }

    // 操作2：扣减库存
    err = stockModel.DecrStock(l.ctx, req.ProductId, req.Num)
    if err != nil {
        return err
    }

    // 操作3：扣减余额
    err = accountModel.DecrBalance(l.ctx, req.UserId, req.Amount)
    if err != nil {
        return err
    }

    return nil
})
```

事务传播：

```
// ServiceContext 中注入事务管理器
type ServiceContext struct {
    Config    config.Config
    Trans     *sqlx.Tx           // 事务管理器
}

// 开启事务
func (s *ServiceContext) BeginTrans() *sqlx.Tx {
    tx, _ := s.Mysql.Begin()
    return tx
}

// 使用事务
tx := l.svcCtx.BeginTrans()
err := l.svcCtx.Trans.Transact(func(session sqlx.Session) error {
    // 使用 tx 执行操作
    _, err := tx.Exec("INSERT INTO orders...", ...)
    return err
})
```

分布式事务（补偿模式）：

```
// 场景：跨服务的事务
func (l *CreateOrderLogic) CreateOrder(req *CreateOrderRequest) error {
    // 1. 本地创建订单（待支付）
    order := &model.Order{
        Status: 0, // 待支付
        Amount: req.Amount,
    }
    _, err := l.svcCtx.OrderModel.Insert(l.ctx, order)
    if err != nil {
        return err
    }

    // 2. 调用支付服务
    payResp, err := l.svcCtx.PayRpc.Pay(l.ctx, &pay.PayRequest{
        OrderId: order.Id,
        Amount:  order.Amount,
    })
    if err != nil {
        // 3. 支付失败，补偿：取消订单
        l.svcCtx.OrderModel.Update(l.ctx, &model.Order{
            Id:      order.Id,
            Status: -1, // 已取消
        })
        return err
    }

    // 4. 支付成功，补偿：更新订单状态
    if payResp.Status == "success" {
        l.svcCtx.OrderModel.Update(l.ctx, &model.Order{
```

```
        Id:    order.Id,
        Status: 1, // 已支付
    })
}

return nil
}
```

事务注意事项：

- 避免长事务，影响数据库性能
- 做好回滚日志记录
- 分布式事务考虑最终一致性

11.6. 错误处理

```
import "github.com/zeromicro/go-zero/core/errorx"

var UserNotFoundError = errorx.New(404, "用户不存在")

if user == nil {
    return nil, UserNotFoundError
}
```

12. 部署与运维

12.1. Docker 部署

```
# 生成 Dockerfile
goctl docker -go demo.go

# 构建运行
docker build -t demo:latest .
docker run -d -p 8888:8888 demo:latest
```

12.2. Kubernetes 部署

```
# 生成 K8s 配置
goctl kube deploy -name demo -namespace default -replicas 3 -o k8s.yaml
```

12.3. 性能测试


```
# HTTP 压测
hey -z 10s -c 100 'http://localhost:8888/user/1'

# gRPC 压测
ghz --insecure --proto=user.proto --call=user.User/GetUser -d '{"id": 1}'
127.0.0.1:8080
```

13. 微服务架构

13.1. 服务拆分原则

- **单一职责**：每个服务专注单一业务领域
- **独立部署**：服务可独立构建、部署、扩展
- **数据隔离**：各服务拥有独立数据库
- **API 优先**：服务间通过 API 通信

13.2. 服务间通信

```
// API 服务调用 RPC
type ServiceContext struct {
    UserRpc user.UserClient
}

resp, _ := l.svcCtx.UserRpc.GetUser(l.ctx, &user.IdRequest{Id: req.Id})
```

14. 调试与实战

14.1. 调试接口

```
curl http://localhost:8888/health # 健康检查
curl http://localhost:8888/debug/routes # 路由列表
curl http://localhost:8888/debug/vars # 运行时信息
```

14.2. 用户注册流程

```
func (l *RegisterLogic) Register(req *types.RegisterRequest) error {
    // 1. 检查重复
    if exists, _ := l.svcCtx.UserModel.ExistsByEmail(l.ctx, req.Email);
    exists {
        return errors.New("邮箱已被注册")
    }
    // 2. 加密密码
    hashedPassword, _ := bcrypt.GenerateFromPassword([]byte(req.Password),
```

```
    bcrypt.DefaultCost)
    // 3. 创建用户
    _, err := l.svcCtx.UserModel.Insert(l.ctx, &model.User{
        Email:    req.Email,
        Password: string(hashAndPassword),
    })
    return err
}
```

14.3. 支付订单流程

```
func (l *PayOrderLogic) PayOrder(req *types.PayOrderRequest) error {
    // 1. 分布式锁
    lock, _ := redis.Lock(l.svcCtx.Redis, fmt.Sprintf("pay:%d",
req.OrderId), 30*time.Second)
    defer lock.Unlock()

    // 2. 事务处理
    return l.svcCtx.Trans.Transact(func(session sqlx.Session) error {
        // 扣款、创建订单
        return nil
    })
}
```

15. 附录

15.1. goctl 命令速查

命令	说明
goctl api new	创建 API 项目
goctl api go	生成 API 代码
goctl api format	格式化 API 文件
goctl rpc new	创建 RPC 项目
goctl rpc go	生成 RPC 代码
goctl model mysql ddl	从 DDL 生成模型
goctl model mysql datasource	从数据库生成模型
goctl model mongo	生成 MongoDB 模型
goctl docker	生成 Dockerfile
goctl kube deploy	生成 K8s 配置

15.2. 常见问题

问题	解决方案
配置无法解密	检查环境变量和加密密钥
编译问题	Go >= 1.18 , 执行 <code>go mod tidy</code>
连接池问题	调整 MaxConns/MaxIdleConns
Context 传递错误	始终使用 <code>l.ctx</code>
并发访问 Map	使用 <code>sync.Map</code> 或加锁

15.3. 最佳实践

- 必用 **RecoverHandler** : 防止 panic 导致服务崩溃
- 合理使用缓存 : 减少数据库压力
- 做好限流熔断 : 保护下游服务
- 使用分布式锁 : 保证并发安全
- 做好日志记录 : 便于问题排查
- 避免 Context 泄漏 : 始终传递 `l.ctx`

16. 总结

go-zero 核心优势 :

特性	说明
开箱即用	API + RPC + DB + Cache 一站式解决方案
效率提升	gocli 脚手架自动生成代码
工程化	内置日志、监控、限流、熔断、链路追踪
高性能	基于 Go 高并发特性优化
易部署	原生支持 Docker 和 Kubernetes

推荐学习路径 : API 服务 → RPC 服务 → 数据库模型 → 服务治理 → 微服务架构